

DB구조와 FAISS구조가 다른 이유

이유는 **DB와 FAISS의 역할이 다르기 때문**이야.

지금 너희 구조에서는:

```
DB
= "어떤 기업/연도/보고서/지수에 속하는 데이터인지" 관리

FAISS
= "후보 문서들 중 어떤 게 질문과 가장 비슷한지" 계산
```

KRX 지수는 본질적으로 **“기업들을 어떻게 묶을 것인가”라는 메타데이터**야. 그래서 DB 쪽에만 새 관계를 추가하면 돼.

예를 들어:

```
KRX 반도체
↓
krx_index_constituents
↓
삼성전자, SK하이닉스, 한미반도체...
```

이건 **검색 대상 기업 목록을 정하는 정보**지, 새로운 벡터 자체를 만드는 정보는 아니야.

그래서 검색할 때는:

```
사용자: KRX 반도체에서 설비투자 찾아줘
↓
DB에서 KRX 반도체 구성 corp_codes 조회
↓
해당 기업 chunk만 후보로 필터
↓
그 후보들만 FAISS 유사도 검색
```

하면 돼.

즉 기존 FAISS는 그대로 써도 돼.

왜 지수별 FAISS를 따로 만들 필요가 없냐면

만약 이렇게 만들면:

```
krx_semiconductor.faiss
krx_bank.faiss
krx_auto.faiss
krx_health.faiss
```

같은 기업이 여러 지수에 들어갈 경우 동일한 벡터를 여러 인덱스에 중복 저장하게 돼.
그러면:

- 저장공간 중복
- 기업 추가할 때 여러 FAISS 갱신
- 지수 구성 변경 때 재구성 필요
- 인덱스 관리 복잡
- 데이터 불일치 위험

이 생겨.

반면 지금 구조는:

```
통합 Vector DB / FAISS 하나
+
DB에서 corp_codes 필터
```

라서 훨씬 단순해.

특히 너희 검색 구조에서는 더더욱 안 바뀌도 돼

너희는 원래도 전체 FAISS에서 먼저 검색하는 구조가 아니고, DB에서 먼저 후보를 좁힌 뒤 임시 `IndexFlatIP` 를 만드는 방식이잖아.

즉 지금도:

```
corp_code = 삼성전자
↓
삼성전자 chunk만 추출
↓
임시 FAISS 검색
```

이었는데,

앞으로는 단지:

```
corp_codes = KRX 반도체 구성기업들
↓
그 기업 chunk들만 추출
↓
임시 FAISS 검색
```

으로 넓어지는 것뿐이야.

그러니까 검색 엔진 자체는 안 바뀌고, 앞단 필터 조건만 1개 기업 → 여러 기업으로 확장되는 것이야.

한 줄로 정리하면:

KRX 지수는 벡터 구조를 바꾸는 정보가 아니라, 어떤 기업의 벡터를 검색 후보로 포함할 지 정하는 필터 정보이기 때문에 DB만 확장하고 FAISS 구조는 그대로 유지하는 게 맞아.

있을 수는 있어. 다만 지금처럼 역할을 분리해두면, 불일치를 '조용히 틀린 결과'로 만들지 않고 명확하게 감지할 수 있어. 핵심은 DB와 FAISS의 "구조"가 같아야 하는 게 아니라, 서로 참조하는 ID와 데이터 범위가 일치해야 한다는 것이야.

예를 들어 DB에는 KRX 반도체 구성기업으로 **삼성전자, SK하이닉스**가 들어 있는데, Vector DB/FAISS에는 삼성전자 벡터만 있고 SK하이닉스 벡터가 없다면:

```
KRX 반도체 선택
↓
corp_codes = [삼성전자, SK하이닉스]
↓
DB 필터
↓
SK하이닉스 chunk 없음
↓
검색 결과가 일부 기업만 반영
```

이런 식으로 **벡터 검색이 불완전해질 수 있어.**

그래서 꼭 체크해야 하는 건 이런 것들이야.

- KRX 구성종목에 포함된 corp_code가 **companies**에 실제 존재하는가
- 그 기업의 **filings/chunks**가 실제로 존재하는가
- searchable chunk에 대응하는 embedding이 있는가
- **vector_id**가 FAISS와 연결되는가
- FAISS **ntotal**과 searchable vector count가 일치하는가

- 특정 지수 선택 시 구성기업 중 데이터가 없는 기업을 명시적으로 표시하는가
- 즉 부작용을 막으려면 검색 전에 이런 검증을 두면 돼.

```
KRX 지수
↓
corp_codes 조회
↓
각 corp_code 데이터 coverage 확인
↓
검색 가능한 기업만 후보에 포함
↓
누락 기업은 warning
↓
FAISS 검색
```

예를 들면 결과에:

```
KRX 반도체 구성기업: 8개
검색 가능 기업: 6개
데이터 미구축: 2개
```

처럼 표시하는 거지.

그래서 **DB와 FAISS가 물리적으로 같은 구조일 필요는 전혀 없고**, 대신 `corp_code → chunk_id`
`→ vector_id` 연결과 coverage가 정확해야 해.

한 줄로 말하면:

구조가 다른 건 문제 없지만, DB가 “있다”고 말하는 데이터가 Vector/FAISS에 실제로 없으면 검색 누락이 생길 수 있으니, coverage와 ID 무결성 검증을 반드시 두면 된다.

. “구조를 통일한다”는 게 무엇을 뜻하느냐에 따라 오히려 더 복잡해질 수 있어.

지금 너희 구조는 사실 역할이 명확해서 좋은 편이야.

```
MySQL/SQLite
= 메타데이터와 관계 관리
  (기업, 지수, 연도, 보고서, corp_code 등)

FAISS
= 벡터 유사도 검색
```

여기서 KRX 지수 구조까지 FAISS 안에 똑같이 복제해서 "통일"하려고 하면, 예를 들어 지수별 인덱스를 따로 만들거나 벡터를 지수별로 중복 저장해야 할 수 있어.

그러면:

```
KRX 반도체 FAISS
KRX 은행 FAISS
KRX 자동차 FAISS
KRX 헬스케어 FAISS
```

처럼 늘어나고, 같은 기업이 여러 지수에 들어가면 벡터도 중복될 수 있어.

그 결과:

- 인덱스 중복
- 업데이트 여러 번
- 지수 정기변경 때 재구성
- 누락/버전 불일치 위험 증가

가 생겨.

오히려 좋은 통일 방식은 **"물리적 구조를 같게 만드는 것"이 아니라 "공통 식별자와 규칙을 통일하는 것"이야.

즉:

```
corp_code
rcept_no
chunk_id
vector_id
```

이 연결은 엄격하게 맞추고,

```
KRX 지수
→ corp_codes
→ 해당 corp_codes의 chunks
→ vector_id
→ FAISS 검색
```

으로 이어지게 하는 거야.

그래서 나는 이렇게 정리하는 게 가장 좋다고 봐:

DB와 FAISS의 저장 구조는 각 역할에 맞게 다르게 유지하되, `corp_code → chunk_id → vector_id` 연결 규칙과 coverage 검증은 하나의 기준으로 통일한다.

즉 "구조 통일"보다 **"ID 체계와 검증 규칙 통일"**이 더 중요해.